

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Operating Systems **COURSE CODE:** CSA04

COURSE OUTCOME COVERED

CO4: Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance.

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:100

Annexure A

INDUSTRY PROBLEM SOLVING TASK

Code of Conduct:

I SIVA SUBRAMANIYAM B/192572089 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

B. Sivasubramaniam.

Annexure A

INDUSTRY PROBLEM SOLVING TASK

S.No	Industry Problem Solving Task Questions	Marks
1	A cloud storage company is facing performance issues due to frequent file access delays. Analyze whether contiguous, linked, or indexed allocation would be most suitable for improving access speed and justify your choice with reasons.	10
2	A data center experiences high disk head movement while processing user requests. Suggest the most appropriate disk scheduling algorithm (FCFS, SSTF, SCAN) to reduce seektime and explain how it improves performance in this scenario.	10
3	An operating system uses a bitmap for free space management but struggles with slow allocation when the disk size increases. Propose an optimization technique to improve the efficiency of bitmap-based allocation and justify your solution.	10
4	A real-time application requires consistent and predictable disk access times. Evaluate whether SCAN or C-SCAN would be more suitable and explain your reasoning based on system requirements.	10
5	A company wants to design a hybrid storage system combining file allocation and disk scheduling techniques. Suggest a suitable combination (allocation + scheduling) for handling large-scale data access efficiently and justify your choice based on performance factors.	10
6	Intelligent File Allocation and Storage Optimization System Industry Context: Cloud storage / Data Centers Design a file-storage simulator for a data center that supports Contiguous, Sequential/Linked, and Indexed allocation. Problem Requirements: - Accept file creation requests with file name and required number of blocks. - Implement all three allocation strategies. - Maintain a dynamic disk-block map. - Support file creation, deletion, extension, and block release. - Detect insufficient storage. - Calculate internal and external fragmentation. - Compare storage utilization and access efficiency.	10

	Recommend the most suitable allocation strategy based on file size and access pattern. Advanced Challenge: Develop an algorithm that automatically selects the allocation strategy for each file based on its size, growth rate, and sequential/random access requirements. Expected Outcome: A storage-management dashboard showing block allocation, fragmentation, utilization, and recommended allocation method.	
7	Unix File System Inode and Buffer Cache Simulator Industry Context: Operating-system kernel / Storage systems Develop a simulator that models the internal operation of a Unix-like file system using inodes and buffer cache. Problem Requirements: - Maintain an inode table containing file metadata. - Implement simulated versions of: - ialloc() - ifree() - namei() - alloc() - free() - Implement pathname resolution using namei(). - Maintain a buffer cache for disk blocks. - Simulate cache hit and cache miss situations. - Track dirty and clean buffers. - Implement buffer replacement when the cache is full. - Calculate cache hit ratio and disk-access reduction. Advanced Challenge: Compare different buffer-cache replacement strategies and determine which provides the best performance for different workloads. Expected Outcome: A simulation showing the complete path: Application → System Call → Inode → Buffer Cache → Disk	10

8	Distributed Network File System with Mount/Unmount Management Industry Context: Enterprise servers / Distributed storage Design a Network File System (NFS)-like simulator where multiple client machines access files stored on a remote server. Problem Requirements: • Create a centralized file server. • Simulate multiple clients. • Implement file creation, opening, reading, writing, and deletion. • Implement mounting and unmounting of remote file systems. • Maintain client-side mount tables. • Handle simultaneous file access. • Simulate server failure and recovery. • Detect invalid mount/unmount operations. • Maintain file consistency between clients and server. Advanced Challenge: Design a mechanism to handle concurrent writes to the same file while minimizing conflicts and maintaining consistency. Expected Outcome: A distributed file-system simulator showing: Client → Network → File Server → File System → Disk	10
9	Intelligent File-System Recovery and Crash Consistency System Industry Context: Database servers / Banking / Mission-critical systems Develop a simulator that investigates what happens when a system crashes during file-system operations. Problem Requirements: • Simulate file creation, deletion, allocation, and modification. • Maintain metadata such as inodes and free-block information. • Generate random system crashes during operations. • Identify inconsistent states after a crash. • Detect: ○ Lost blocks ○ Duplicate block allocation ○ Invalid inode references ○ Orphan files	10

	○ Corrupted free-space information • Implement a recovery mechanism. • Restore the file system to a consistent state. Advanced Challenge: Compare different recovery approaches and calculate data loss, recovery time, and consistency level. Expected Outcome: A File System Health and Recovery Tool that reports: Normal State → Crash → Corruption Detection → Recovery → Verified File System	
10	High-Performance I/O and File-System Optimization Simulator Industry Context: Cloud platforms / Video streaming / High-performance computing Develop a simulator that optimizes file-system performance for a workload involving thousands of file operations. Problem Requirements: • Generate read/write requests from multiple applications. • Simulate different file sizes and access patterns. • Implement file allocation and disk-block management. • Simulate the I/O subsystem and request queues. • Include buffer caching. • Measure: ○ File access time ○ Disk I/O operations ○ Cache hit ratio ○ Throughput ○ Storage utilization ○ Fragmentation • Compare sequential and random file access. • Identify system bottlenecks. Advanced Challenge: Build an adaptive optimization algorithm that dynamically changes caching, allocation, and I/O scheduling decisions according to the workload.Expected Outcome: A performance analyzer that recommends the optimal configuration for workloads such as: • Video streaming	10

	• Database transactions • Web-server logs • Large scientific datasets • Cloud file storage	
--	---	--

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding ; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

1. A cloud storage company is facing performance issues due to frequent file access delays. Analyze whether contiguous, linked, or indexed allocation would be most suitable for improving access speed and justify your choice with reasons.

Answer:

File allocation is the method used by an operating system to assign disk blocks to files. The three common methods are **contiguous allocation, linked allocation, and indexed allocation.**

Contiguous Allocation

In contiguous allocation, all blocks of a file are stored continuously.

Example:

File A → 10 → 11 → 12 → 13 → 14

Advantages:

- Very fast sequential access.
- Very fast direct access.
- Low overhead.
- Simple to implement.

Disadvantages:

- Causes external fragmentation.
- File growth is difficult.

- Requires consecutive free blocks.

Linked Allocation

In linked allocation, file blocks can be located anywhere on the disk. Each block contains a pointer to the next block.

5 → 18 → 25 → 40 → NULL

Advantages:

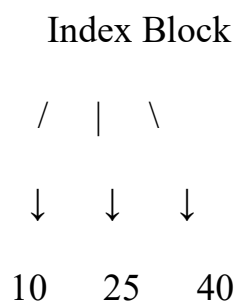
- No external fragmentation.
- Easy file growth.
- Any available block can be used.

Disadvantages:

- Random access is slow.
- Pointer overhead is required.
- Blocks may be scattered across the disk.

Indexed Allocation

Indexed allocation uses an index block containing addresses of the file's data blocks.



Advantages:

- Supports direct and random access.

- Allows flexible file growth.
- Does not require consecutive blocks.
- Suitable for large files.

Disadvantages:

- Requires additional index-block storage.
- Large files may require multiple index levels.

Comparison

Feature	Contiguous	Linked	Indexed
Sequential Access	Excellent	Good	Good
Random Access	Excellent	Poor	Excellent
File Growth	Difficult	Easy	Easy
External Fragmentation	Yes	No	No
Flexibility	Low	High	High

Conclusion

For the given cloud-storage problem, **indexed allocation is the most suitable overall choice**. Contiguous allocation provides excellent speed but has fragmentation and file-growth problems. Linked allocation provides flexibility but has slower random access. Indexed allocation provides a good balance between **access speed, flexibility, scalability, and storage efficiency**.

2. A data center experiences high disk head movement while processing user requests. Suggest the most appropriate disk scheduling algorithm (FCFS, SSTF, SCAN) to reduce seek time and explain how it improves performance.

Answer:

Disk scheduling determines the order in which disk I/O requests are processed.

The three algorithms given are **FCFS, SSTF, and SCAN**.

FCFS

First-Come, First-Served services requests in their arrival order.

Example:

98 → 183 → 37 → 122 → 14

Advantages:

- Simple.
- Fair.
- Easy to implement.

Disadvantage:

- Can cause large disk-head movements.

SSTF

Shortest Seek Time First selects the request closest to the current disk-head position.

For example, if the current head is at 50:

Requests: 20, 45, 70, 100

SSTF selects 45 because it is closest to 50.

Advantages:

- Reduces average seek time.
- Reduces unnecessary head movement.

Disadvantage:

- Requests far away may experience starvation.

SCAN

SCAN moves the disk head in one direction while servicing requests and then reverses direction.

→ → → → →

← ← ← ← ←

It is also called the **elevator algorithm**.

Recommended Algorithm

SCAN is the most appropriate choice for a data center where high disk-head movement is a problem.

SCAN reduces unnecessary movement by servicing requests in an organized direction. It also provides better fairness than SSTF.

Conclusion

SCAN is **recommended** because it provides a good balance between reduced seek time, fairness, and predictable performance. It is more suitable for a busy data center than FCFS, while avoiding some starvation problems associated with SSTF.

3. An operating system uses a bitmap for free-space management but struggles with slow allocation when the disk size increases. Propose an optimization technique to improve the efficiency of bitmap-based allocation and justify your solution.

Answer:

A bitmap is a free-space management technique in which every disk block is represented by a bit.

0 = Free

1 = Allocated

Example:

Blocks: 0 1 2 3 4 5 6 7

Bitmap: 1 1 0 0 1 0 1 0

Here, blocks 2, 3, 5, and 7 are free.

Problem

As the disk becomes larger, the bitmap also becomes larger. Searching the bitmap bit by bit can take more time.

Proposed Optimization: Hierarchical Bitmap

A **hierarchical bitmap** can be used to improve allocation efficiency.

Instead of searching the entire bitmap, the disk can be divided into groups.

Level 1



Groups containing free blocks



Level 2



Individual block bitmap

The operating system first checks the higher-level bitmap to identify a group containing free space. It then searches only that group.

Other Optimization Techniques

- Word-level bitmap searching.
- Caching frequently accessed bitmap regions.
- Maintaining free-block counters.
- Using hierarchical free-space information.

Advantages

1. Reduces bitmap search time.
2. Avoids scanning the entire disk.
3. Works efficiently for large disks.
4. Reduces allocation overhead.

5. Improves storage-management performance.

Conclusion

A **hierarchical bitmap** is a suitable optimization because it reduces the amount of bitmap information that must be searched before finding a free block. It is particularly useful when the disk size is large.

4. A real-time application requires consistent and predictable disk access times. Evaluate whether SCAN or C-SCAN would be more suitable and explain your reasoning based on system requirements.

Answer:

SCAN and C-SCAN are disk scheduling algorithms designed to reduce unnecessary disk-head movement.

SCAN

SCAN moves the disk head in one direction, services requests, reaches the end, and then reverses direction.

→ → → → →

← ← ← ← ←

C-SCAN

C-SCAN services requests in only one direction. After reaching the end, the disk head returns to the beginning and starts servicing requests again.

→ → → → → END

↓

START → → → →

Comparison

Feature	SCAN	C-SCAN
Direction	Both directions	One direction
Fairness	Good	Very Good
Waiting Time	Variable	More Uniform
Predictability	Good	Better
Suitable for Real-Time Access	Good	Very Good

Recommended Algorithm

C-SCAN is more suitable for the given real-time application.

The reason is that C-SCAN provides a more uniform service pattern. Requests are serviced in one direction, which makes waiting times more predictable.

Conclusion

For a real-time application where **consistent and predictable disk access times** are important, **C-SCAN is recommended over SCAN**.

5. A company wants to design a hybrid storage system combining file allocation and disk scheduling techniques. Suggest a suitable combination (allocation + scheduling) for handling large-scale data access efficiently and justify your choice based on performance factors.

Answer:

A hybrid storage system can combine a file allocation technique with a disk scheduling algorithm to improve overall storage performance.

Recommended Combination

Indexed Allocation + C-SCAN

Indexed Allocation

Indexed allocation provides:

- Efficient random access.
- Flexible file growth.
- Support for large files.
- No requirement for consecutive blocks.

C-SCAN

C-SCAN provides:

- Predictable request servicing.
- More uniform waiting time.
- Reduced unnecessary disk-head movement.
- Good performance for large request queues.

Proposed Architecture

Application



File System



Indexed Allocation



I/O Request Queue



C-SCAN Scheduler



Disk

Why This Combination?

Large-scale systems may contain many files and a large number of simultaneous I/O requests.

Indexed allocation manages the location of file blocks efficiently, while C-SCAN organizes disk requests in a predictable manner.

Performance Factors

Factor	Indexed + C-SCAN
Random Access	High
File Growth	Good
Large Files	Good
Request Management	Efficient
Predictability	High
Scalability	High

Conclusion

Indexed allocation combined with C-SCAN is a suitable hybrid solution for large-scale storage because it combines flexible file management with predictable disk scheduling.

6. Intelligent File Allocation and Storage Optimization System

Answer:

The objective is to design a file-storage simulator that supports **Contiguous, Linked, and Indexed allocation**.

Main Modules

1. File creation.
2. File deletion.
3. File extension.

4. Block release.
5. Disk-block management.
6. Fragmentation calculation.
7. Storage-utilization analysis.
8. Access-efficiency analysis.
9. Allocation-strategy recommendation.

System Architecture

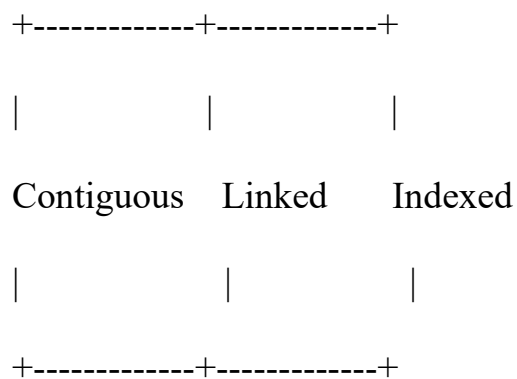
User



File Request



Storage Manager



Disk Block Map



Performance Analyzer



Recommendation

File Creation

The system accepts:

File Name

File Size

Number of Blocks

Access Pattern

Growth Rate

Example:

File Name: video.mp4

Blocks: 100

Access: Sequential

Growth Rate: Low

File Deletion

When a file is deleted:

1. File metadata is removed.
2. Allocated blocks are released.
3. Disk-block map is updated.
4. Free-space information is updated.

File Extension

When a file grows, the simulator attempts to allocate additional blocks.

Contiguous allocation may require continuous blocks.

Linked allocation can add blocks anywhere.

Indexed allocation can add new block addresses to the index.

Fragmentation

The system calculates:

- Internal fragmentation.
- External fragmentation.

Storage Utilization

Formula:

Storage Utilization =

Used Storage / Total Storage $\times$ 100

Access Efficiency

The simulator compares the access performance of each allocation strategy.

Automatic Selection

A decision algorithm can be:

IF sequential access AND stable file size

 Choose Contiguous

ELSE IF high growth rate

 Choose Linked

ELSE IF random access

 Choose Indexed

ELSE

Compare performance scores

Select best method

Expected Dashboard

```
+-----+
| STORAGE MANAGEMENT DASHBOARD |
+-----+
| Total Blocks:    1000    |
| Used Blocks:     650    |
| Free Blocks:     350    |
| Utilization:     65%    |
| Fragmentation:   12%    |
| Recommended Method: Indexed |
+-----+
```

Conclusion

The proposed simulator provides a practical method for comparing allocation techniques and automatically selecting the most suitable strategy based on file size, growth rate, and access pattern.

7. Unix File System Inode and Buffer Cache Simulator

Answer:

The purpose of this simulator is to model the internal operation of a Unix-like file system using **inodes and buffer cache**.

Inode

An inode stores file metadata such as:

- File type.
- File size.
- File permissions.
- File ownership.
- Timestamps.
- Data-block addresses.

Required Functions

The simulator should implement:

`ialloc()`

`ifree()`

`namei()`

`alloc()`

`free()`

Function Purpose

ialloc(): Allocates an inode for a new file.

ifree(): Releases an inode when a file is deleted.

namei(): Resolves a pathname and identifies the corresponding file.

alloc(): Allocates a disk block.

free(): Releases a disk block.

Buffer Cache

Buffer cache stores frequently accessed disk blocks in memory.

If the requested block exists in the cache:

Cache Hit

If it does not exist:

Cache Miss



Disk Access



Load Block into Cache

Cache Hit Ratio

Cache Hit Ratio =

Cache Hits / Total Requests $\times 100$

For example:

Cache Hits = 800

Total Requests = 1000

Hit Ratio = $800/1000 \times 100$

= 80%

Dirty and Clean Buffers

A **clean buffer** contains data that matches the disk.

A **dirty buffer** contains modified data that has not yet been written to disk.

Buffer Replacement

When the cache is full, a buffer must be removed.

Possible strategies include:

- FIFO.
- LRU.
- Clock.

Expected Flow

Application



System Call



Inode



Buffer Cache



Disk

Conclusion

The simulator demonstrates how Unix-like file systems manage metadata and reduce disk access using buffer caching.

8. Distributed Network File System with Mount/Unmount Management

Answer:

A Network File System-like simulator allows multiple client computers to access files stored on a remote file server.

Architecture

Client 1 —┐

Client 2 —┴—→ Network → File Server → File System → Disk

Client 3 —┐

File Server

The centralized server manages:

- Files.
- Directories.
- Storage.
- Client requests.
- File consistency.

Client Operations

The simulator supports:

1. File creation.
2. File opening.
3. File reading.
4. File writing.
5. File deletion.
6. Mounting.
7. Unmounting.

Mount Operation

Mounting connects the client's file system to the remote file system.

Client



Mount Request



Server Validation



Mount Successful

Unmount Operation

Before unmounting, the system should check whether the file system is currently being used.

Mount Table

Each client maintains a mount table.

Example:

Client	Remote Server	Mount Point	Status
C1	Server	/remote	Mounted
C2	Server	/data	Mounted

Concurrent Access

Multiple clients may access the same file.

To maintain consistency, the system can use:

- Read locks.
- Write locks.
- File-level locking.
- Version control.

Concurrent Write Example

Client A → Write File X

Client B → Write File X

The system can give a write lock to one client.

Client A

↓

Write Lock

↓

Write

↓

Unlock

↓

Client B

Server Failure

The system should detect server failure and restore normal operation after recovery.

Conclusion

The simulator demonstrates distributed file access, mount/unmount operations, concurrency, server failure, recovery, and consistency management.

9. Intelligent File-System Recovery and Crash Consistency System

Answer:

A file-system crash can occur while files are being created, deleted, allocated, or modified. The purpose of this simulator is to identify inconsistent states and restore the file system to a consistent state.

Operations to Simulate

- File creation.
- File deletion.
- Block allocation.
- File modification.

Metadata

The simulator maintains:

- Inode information.
- File information.
- Directory information.
- Free-block information.

Possible Errors

1. Lost Blocks

A block is marked as allocated but is not referenced by any file.

2. Duplicate Block Allocation

Two files incorrectly reference the same block.

File A → Block 20

File B → Block 20

3. Invalid Inode Reference

An inode points to an invalid or unavailable block.

4. Orphan File

A file exists in the inode structure but has no valid directory entry.

5. Corrupted Free-Space Information

The free-space map does not correctly represent the actual disk state.

Recovery Process

Normal State



System Crash



Corruption Detection



Metadata Analysis



Recovery



Consistency Check



Verified File System

Recovery Metrics

The simulator can calculate:

- Data loss.
- Recovery time.
- Number of corrupted blocks.
- Number of recovered blocks.
- Consistency level.

Conclusion

The recovery system helps identify file-system corruption and restores the system to a consistent state. It is particularly important for mission-critical systems such as databases and banking systems.

10. High-Performance I/O and File-System Optimization Simulator

Answer:

The objective is to develop a simulator capable of optimizing file-system performance for thousands of file operations.

System Architecture

Applications



Read/Write Requests



I/O Request Queue



File Allocation



Buffer Cache



Disk Scheduling



Storage Device

Input Workloads

The simulator can generate:

- Read requests.
- Write requests.
- Different file sizes.
- Sequential access.
- Random access.

I/O Subsystem

The I/O subsystem manages requests generated by different applications.

Application 1 —┐

Application 2 —┤—→ Request Queue → Scheduler → Disk

Application 3 —┐

Buffer Caching

Frequently accessed blocks are stored in memory to reduce disk operations.

Performance Measurements

The simulator measures:

1. File Access Time

Time required to complete a file request.

2. Disk I/O Operations

Number of read/write operations performed by the disk.

3. Cache Hit Ratio

Hit Ratio =

$\text{Cache Hits} / \text{Total Requests} \times 100$

4. Throughput

Amount of data processed during a given period.

5. Storage Utilization

Utilization =

$\text{Used Storage} / \text{Total Storage} \times 100$

6. Fragmentation

Measures how storage space is divided into separate regions.

Sequential vs Random Access

Sequential access:

10 → 11 → 12 → 13 → 14

Data is accessed in order.

Random access:

10 → 50 → 15 → 80 → 25

Data is accessed at different locations.

Sequential access generally benefits from contiguous block placement, while random access benefits from efficient indexing.

Bottleneck Identification

The simulator should identify whether performance is limited by:

- Disk access.
- CPU processing.
- Cache misses.
- Request queues.
- Fragmentation.
- Poor allocation.
- Disk scheduling.

Adaptive Optimization

The advanced system can dynamically change:

Caching

+

Allocation

+

I/O Scheduling

+

Access Strategy

according to workload conditions.

Workload Recommendations

Workload	Suitable Approach
Video Streaming	Sequential-friendly allocation + efficient scheduling
Database Transactions	Indexed allocation + suitable scheduling
Web Logs	Sequential/linked or indexed allocation
Scientific Datasets	Contiguous or indexed allocation
Cloud File Storage	Indexed allocation + C-SCAN

Conclusion

The high-performance I/O simulator combines file allocation, disk management, buffer caching, and scheduling. By measuring access time, throughput, cache hit ratio, utilization, and fragmentation, it can identify bottlenecks and recommend an efficient configuration for different workloads.